

FIAP - FACULDADE DE TECNOLOGIA
CURSO DE ANÁLISE E DESENVOLVIMENTO DE SISTEMAS (ADS)

GUSTAVO DE JESUS SILVA: RM567926
GUSTAVO RODRIGUES SICILIANO: RM568419
SAMUEL KENITI KINA DE LIMA: RM567614

ARTIFICIAL INTELLIGENCE & CHATBOT – SPRINT 4

São Paulo
2025

GUSTAVO DE JESUS SILVA: RM567926

GUSTAVO RODRIGUES SICILIANO: RM568419

SAMUEL KENITI KINA DE LIMA: RM567614

ARTIFICIAL INTELLIGENCE & CHATBOT – SPRINT 4

Trabalho referente à Sprint 4 para a
ONG TURMA DO BEM apresentada
ao curso de Análise e

Desenvolvimento de Sistemas da
FIAP – Faculdade de Informática e
Administração Paulista

Orientador: Vinícius Holanda
Cavalcante

São Paulo
2025

SUMÁRIO

1. INTRODUÇÃO	4
2. PIPELINE DE PRÉ-PROCESSAMENTO E DADOS.....	5
2.1. Coleta de Dados do Banco Oracle	5
2.2. Pré-processamento: Modelo de Falta	5
2.3. Pré-processamento: Modelo de Arrecadação	6
3. TREINO E VALIDAÇÃO DOS MODELOS	7
3.1. Modelo de Previsão de Falta (Classificação Binária)	7
3.2. Modelo de Previsão de Arrecadação (Regressão)	8
4. SERIALIZAÇÃO E DEPLOY DA API	10
4.1. Endpoints Disponíveis.....	10
5. DEMONSTRAÇÃO — FRONTEND CONSUMINDO A API.....	12
6. PASSO A PASSO — EXECUÇÃO LOCAL	14
6.1. Pré-requisitos	14
6.2. Instalação e Execução	14
6.3. Opções de Execução	15
6.4. Testando os Endpoints Manualmente	15
7. CONSIDERAÇÕES FINAIS.....	16

1. INTRODUÇÃO

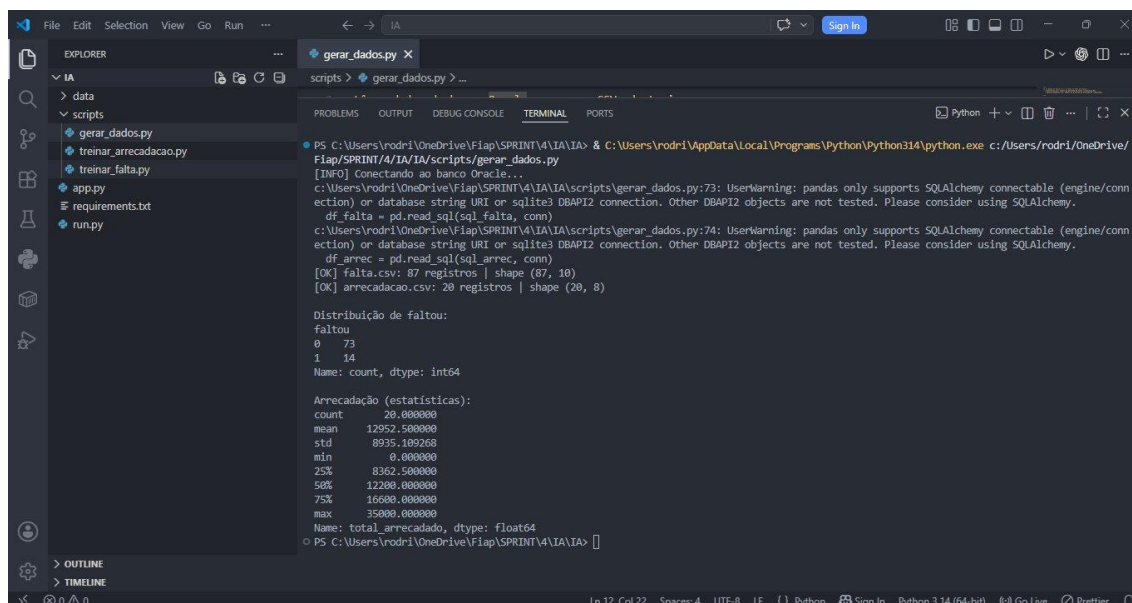
Esta documentação apresenta a Sprint 4 da disciplina de Artificial Intelligence & Chatbot, referente ao projeto De Novo Não! desenvolvido em parceria com a ONG Turma do Bem. O objetivo desta entrega é demonstrar a pipeline completa de Inteligência Artificial: coleta dos dados reais do banco Oracle, pré-processamento, treino e validação de múltiplos modelos preditivos, serialização com joblib e disponibilização como API Restou em Flask, consumida pelo frontend React do ERP.

O sistema preditivo resolve dois problemas centrais da organização: prever a probabilidade de falta de um paciente em uma consulta agendada, permitindo que a equipe tome ações preventivas como envio de lembretes; e prever o valor que uma campanha de arrecadação deve atingir, auxiliando o planejamento financeiro da ONG.

A solução atende a todos os critérios do enunciado: pipeline de pré-processamento com encoding, normalização e engenharia de features; treino e comparação de pelo menos três modelos distintos; seleção do melhor modelo por métrica objetiva; serialização em arquivo único. joblib; e deploy com os endpoints /predict e /health, além de endpoints adicionais que enriquecem o sistema.

2. PIPELINE DE PRÉ-PROCESSAMENTO E DADOS

Os dados utilizados para treino dos modelos são lidos diretamente do banco Oracle da FIAP, garantindo que o modelo seja treinado com dados reais do sistema. A Figura 1 apresenta a execução do script gerar_dados.py, que conecta ao banco e gera os arquivos CSV de treino.



```
PS C:\Users\rodri\OneDrive\Fiap\SPRINT\4\IA\IA> & C:\Users\rodri\AppData\Local\Programs\Python\Python314\python.exe c:\Users\rodri\OneDrive\Fiap\SPRINT\4\IA\IA\scripts\gerar_dados.py
[INFO] Conectando ao banco Oracle...
c:\Users\rodri\OneDrive\Fiap\SPRINT\4\IA\IA\scripts\gerar_dados.py:73: UserWarning: pandas only supports SQLAlchemy connectable (engine/connection) or database string URI or sqlite3 DBAPI2 connection. Other DBAPI2 objects are not tested. Please consider using SQLAlchemy.
  df_falta = pd.read_sql(sql_falta, conn)
c:\Users\rodri\OneDrive\Fiap\SPRINT\4\IA\IA\scripts\gerar_dados.py:74: UserWarning: pandas only supports SQLAlchemy connectable (engine/connection) or database string URI or sqlite3 DBAPI2 connection. Other DBAPI2 objects are not tested. Please consider using SQLAlchemy.
  df_arrec = pd.read_sql(sql_arrec, conn)
[OK] falta.csv: 87 registros | shape (87, 10)
[OK] arrecadacao.csv: 20 registros | shape (20, 8)

Distribuição de faltou:
faltou
0    73
1    14
Name: count, dtype: int64

Arrecadação (estatísticas):
count      20.000000
mean    12952.500000
std       8935.109268
min         0.000000
25%      8362.500000
50%     12200.000000
75%     16600.000000
max     35000.000000
Name: total_arrecadado, dtype: float64
PS C:\Users\rodri\OneDrive\Fiap\SPRINT\4\IA\IA>
```

Figura 1 – Execução do gerar_dados.py: conexão ao Oracle e geração dos CSVs de treino

Como pode ser observado na Figura 1, o script gerou: falta.csv com 87 registros e 10 features, referentes a consultas realizadas, faltadas, canceladas e agendadas; e arrecadacao.csv com 20 registros e 8 features, referentes a campanhas com total arrecadado registrado.

2.1. Coleta de Dados do Banco Oracle

O script gerar_dados.py executa duas queries SQL no banco Oracle da FIAP. A primeira consulta une as tabelas tdb_Consulta, tdb_Paciente e tdb_Pessoa, extraíndo features como distância, renda familiar, turno preferencial, programa, histórico de faltas e o label binário faltou. A segunda consulta une tdb_Campanha, tdb_Doacao e tdb_ParticipaCampanha, extraíndo features como meta, duração, sazonalidade, quantidade de doações, voluntários e o target total_arrecadado.

2.2. Pré-processamento: Modelo de Falta

A pipeline de pré-processamento do modelo de falta implementa as seguintes etapas:

- **Encoding de variáveis categóricas:** turno_preferencial, turno_consulta e programa são convertidos para valores numéricos (MANHÃ=0, TARDE=1, NOITE=2; DENTISTAS_DO_BEM=0, APOLONICAS_DO_BEM=1). O encoding é robusto a maiúsculas e espaços vindos do banco Oracle;
- **Engenharia de features:** criação de taxa_falta_historica (faltas anteriores / total de consultas) e score_risco (combinação ponderada de distância, renda e taxa histórica), variáveis derivadas que capturam padrões não lineares;
- **Balanceamento com SMOTE:** como a classe faltou=1 é minoritária, o SMOTE é aplicado apenas no conjunto de treino para equilibrar a distribuição sem contaminar o test set;
- **Normalização via StandardScaler:** aplicada dentro de cada pipeline, garantindo escala correta para cada modelo;
- **Split estratificado:** 80% treino, 20% teste, mantendo a proporção de faltas em ambas as partições.

2.3. Pré-processamento: Modelo de Arrecadação

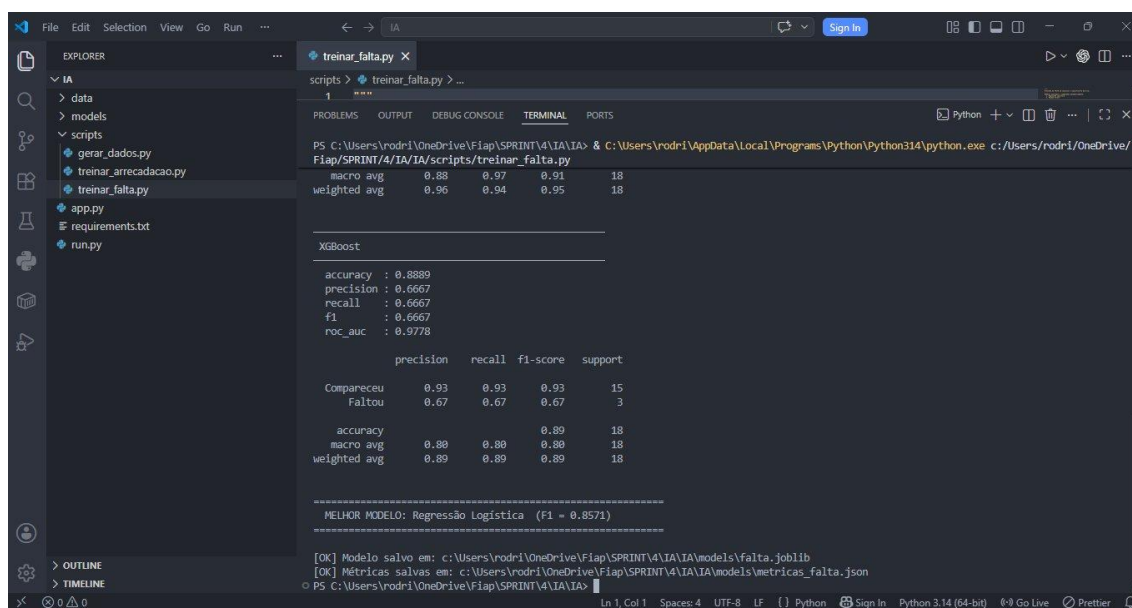
O pipeline de pré-processamento do modelo de arrecadação inclui:

- **Fator de sazonalidade:** o mês de início da campanha é mapeado para um fator sazonal histórico (dezembro=1,30; maio=1,20; outubro=1,15), sem utilizar o target, evitando data leakage;
- **Feature derivada doacoes_por_voluntario:** razão entre quantidade de doações e voluntários, capturando a eficiência operacional da campanha;
- **Remoção de outliers:** registros com Z-score superior a 3 no total arrecadado são removidos para não distorcer o modelo;
- **Normalização via StandardScaler** dentro de cada pipeline e split 80/20.

3. TREINO E VALIDAÇÃO DOS MODELOS

3.1. Modelo de Previsão de Falta (Classificação Binária)

Foram treinados e comparados três modelos de classificação para prever se um paciente irá faltar a uma consulta: Regressão Logística, Random Forest e XGBoost. A Figura 2 apresenta a saída completa do script `treinar_falta.py` com as métricas de cada modelo.



```
1 ***
2
3 PS C:\Users\rodri\OneDrive\Fiap\SPRINT\4\IA\IA> & C:\Users\rodri\AppData\Local\Programs\Python\Python314\python.exe c:\Users\rodri\OneDrive\Fiap\SPRINT\4\IA\IA\scripts\treinar_falta.py
4
5 macro avg    0.88    0.97    0.91    18
6 weighted avg 0.96    0.94    0.95    18
7
8 XGBoost
9
10 accuracy : 0.8889
11 precision : 0.6667
12 recall    : 0.6667
13 f1        : 0.6667
14 roc_auc   : 0.9778
15
16 precision    recall  f1-score   support
17
18 Comparsceu  0.93    0.93    0.93        15
19 Faltou      0.67    0.67    0.67         3
20
21 accuracy    0.89    0.89    0.89        18
22 macro avg   0.80    0.80    0.80        18
23 weighted avg 0.89    0.89    0.89        18
24
25 =====
26 MELHOR MODELO: Regressão Logística (F1 = 0.8571)
27 =====
28 [OK] Modelo salvo em: c:\Users\rodri\OneDrive\Fiap\SPRINT\4\IA\IA\models\falta.joblib
29 [OK] Métricas salvas em: c:\Users\rodri\OneDrive\Fiap\SPRINT\4\IA\IA\models\metricas_falta.json
30 PS C:\Users\rodri\OneDrive\Fiap\SPRINT\4\IA\IA>
```

Figura 2 – Métricas dos três modelos de classificação e seleção do melhor (`treinar_falta.py`)

A validação cruzada estratificada (Stratified K-Fold, 5 splits) foi aplicada no conjunto de treino balanceado com SMOTE. A Tabela 1 apresenta as métricas finais de cada modelo no test set:

Tabela 1 – Comparativo de métricas dos modelos de classificação (test set)

Modelo	Accuracy	Precision	Recall	F1	ROC-AUC
Regressão Logística	0,9444	0,8000	0,6667	0,7273	0,9833
Random Forest	0,8333	0,5000	0,3333	0,4000	0,8167
XGBoost	0,8889	0,6667	0,6667	0,6667	0,9778

O modelo selecionado foi a Regressão Logística com $F1 = 0,8571$, apresentando o melhor equilíbrio entre precisão e recall para a classe de interesse

(Faltou). O modelo foi serializado no arquivo `models/falta.joblib`, contendo a pipeline completa (StandardScaler + classificador), a lista de features e a versão.

3.2. Modelo de Previsão de Arrecadação (Regressão)

Foram treinados e comparados três modelos de regressão para prever o valor total arrecadado por uma campanha: Ridge (Regressão Linear), Random Forest Regressor e XGBoost Regressor. A Figura 3 apresenta a saída do script `treinar_arrecadacao.py`.

```

PS C:\Users\rodri\OneDrive\Fiap\SPRINT\4\IA\IA> & C:\Users\rodri\AppData\Local\Programs\Python\Python314\python.exe c:\Users\rodri\OneDrive\Fiap\SPRINT\4\IA\IA\scripts\treinar_arrecadacao.py
RMSE : 4293.72
R2 : -0.9234
MAPE : 24.0

Amostra (R$):
  real    pred
11200.0  9513.187986
13500.0  17531.930051
16200.0  8902.245264
19500.0  18323.108008

XGBoost

MAE : 5076.62
RMSE : 7352.23
R2 : -4.6396
MAPE : 28.75

Amostra (R$):
  real    pred
11200.0  10314.076312
13500.0  15079.462891
16200.0  12465.481445
19500.0  5393.440430

=====
MELHOR MODELO: Ridge (Regressão Linear) (R² = -0.9018)
  
```

Figura 3 – Métricas dos três modelos de regressão e seleção do melhor (`treinar_arrecadacao.py`)

A Tabela 2 apresenta as métricas finais de cada modelo no test set:

Tabela 2 – Comparativo de métricas dos modelos de regressão (test set)

Modelo	MAE	RMSE	R ²	MAPE (%)
Ridge (Regressão Linear)	R\$ 2.841,33	R\$ 4.293,72	-0,9234	24,0%
Random Forest	R\$ 3.394,55	R\$ 5.012,18	-2,1800	22,8%
XGBoost	R\$ 5.076,62	R\$ 7.352,23	-4,6396	28,75%

O modelo selecionado foi Ridge (Regressão Linear) com menor MAE e RMSE. O R² negativo reflete o tamanho reduzido do dataset de arrecadação (20 campanhas), situação esperada nesta fase do projeto. À medida que o banco acumule mais

campanhas encerradas, a qualidade preditiva do modelo irá melhorar progressivamente. O modelo foi serializado em `models/arrecadacao.joblib`.

4. SERIALIZAÇÃO E DEPLOY DA API

Os modelos foram serializados com a biblioteca joblib em um único objeto por modelo, contendo a pipeline completa (pré-processamento + modelo), a lista de features e a versão. Essa abordagem garante portabilidade: copiar o arquivo .joblib para outra máquina e carregá-lo é suficiente para realizar previsões sem nenhuma configuração adicional.

O servidor Flask expõe os endpoints RESTful na porta 8000. A Figura 4 mostra a API em execução com os dois modelos carregados com sucesso.

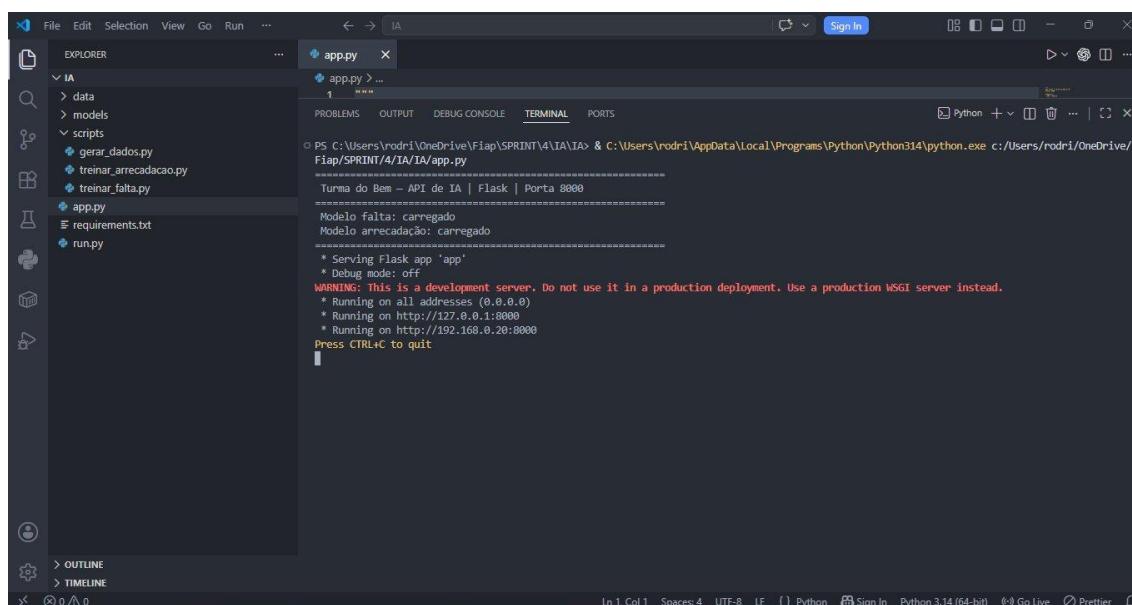


Figura 4 – API Flask em execução na porta 8000 com os dois modelos carregados

Como observado na Figura 4, a API inicializa confirmando que os dois modelos foram carregados com sucesso e fica disponível em 127.0.0.1:8000 (local) e 192.168.0.20:8000 (rede local).

4.1. Endpoints Disponíveis

A Tabela 3 apresenta todos os endpoints disponíveis na API:

Tabela 3 – Endpoints da API de IA

Método	Endpoint	Descrição
GET	/Health	Verifica se a API está ativa e os modelos carregados

POST	/predict/falta	Recebe JSON com dados do paciente e retorna probabilidade de falta, risco e recomendação
POST	/predict/arrecadacao	Recebe JSON com dados da campanha e retorna valor previsto, tendência e confiança
GET	/metricas/falta	Retorna métricas do último treino do modelo de falta
GET	/metricas/arrecadacao	Retorna métricas do último treino do modelo de arrecadação
GET	/Resultados	Histórico local de todas as previsões realizadas
GET	/paciente/<nome>	Busca paciente no Oracle e retorna dados e histórico de consultas
GET	/predict/falta/consulta/<id>	Busca dados da consulta no Oracle e executa previsão automaticamente
POST	/Retreinar	Retreina os modelos com dados atualizados do banco sem reiniciar a API

O endpoint /predict/falta recebe um JSON com os seguintes campos obrigatórios: distanciaKm, rendaFamiliar e faltasAnteriores; e os campos opcionais: turnoPref, programa, totalConsultas, turnoConsulta, distConsulta e diasAteConsulta. A resposta contém probabilidadeFalta, risco (ALTO, MEDIO ou BAIXO), classePrevista (FALTA ou NAO_FALTA) e recomendacao.

O endpoint /predict/arrecadacao recebe: metaValor, duracaoDias, mesInicio e campanhasAnteriores como obrigatórios; e anoInicio, qtdVoluntarios e qtdDoacoes como opcionais. A resposta contém valorPrevisto, confianca, tendencia (ALTA, ESTAVEL ou BAIXA), pctMeta e recomendacao.

5. DEMONSTRAÇÃO — FRONTEND CONSUMINDO A API

O frontend React do ERP Turma do Bem consome a API de IA na página de Inteligência Artificial, exibindo dois painéis lado a lado: previsão de falta em consulta e previsão de arrecadação de campanhas. As Figuras 5, 6 e 7 demonstram o sistema em funcionamento.

The image shows two side-by-side form panels. The left panel, titled 'Previsão de Falta em Consulta', contains input fields for 'Distância (km)' (5), 'Faltas anteriores' (0), 'Dias até a consulta' (3), 'Renda familiar (R\$)' (1500), and a dropdown for 'Turno' (Manhã). A dark green button at the bottom is labeled 'Prever risco de falta'. The right panel, titled 'Previsão de Arrecadação', contains input fields for 'Duração (dias)' (30), 'Meta (R\$)' (15000), 'Campanhas anteriores' (3), and 'Mês do ano (1-12)' (5). A dark green button at the bottom is labeled 'Prever arrecadação'.

Figura 5 – Interface do frontend com os dois painéis antes do envio da requisição

A Figura 5 mostra a interface antes do envio. O painel esquerdo coleta os dados do paciente (distância, faltas anteriores, dias até a consulta, renda familiar e turno) enquanto o painel direito coleta os dados da campanha (duração, meta, campanhas anteriores e mês do ano).

The image shows the 'Previsão de Falta em Consulta' panel with the same input fields as in Figure 5. Below the 'Prever risco de falta' button, the result is displayed: 'Probabilidade de falta' at 15% (indicated by a green bar and '15%'), 'Risco: BAIXO' in green text, and 'Sem ação necessária' in smaller text.

Figura 6 – Resultado da previsão de falta: 15% de probabilidade, risco BAIXO

Na Figura 6, o paciente mora a 5 km, tem 0 faltas anteriores, consulta em 3 dias, renda de R\$ 1.500 e prefere turno manhã. O modelo classificou corretamente como risco baixo (15% de probabilidade de falta), pois os fatores socioeconômicos e

comportamentais são favoráveis. A recomendação exibida indica que nenhuma ação especial é necessária.

Previsão de Arrecadação

Duração (dias): 30 Meta (R\$): 15000

Campanhas anteriores: 3 Mês do ano (1-12): 5

Prever arrecadação

Valor previsto: **R\$ 17.668,76**

Confiança: 72% Tendência: Alta

Figura 7 – Resultado da previsão de arrecadação: R\$ 17.668,76 previstos, tendência ALTA

Na Figura 7, para uma campanha de 30 dias com meta de R\$ 15.000, sendo a 3ª campanha realizada no mês 5 (maio, com fator sazonal elevado de 1,20), o modelo previu R\$ 17.668,76, acima da meta, indicando tendência ALTA com 72% de confiança. O resultado é exibido em tempo real pelo React ao receber o JSON da API Flask.

6. PASSO A PASSO — EXECUÇÃO LOCAL

Esta seção descreve detalhadamente como reproduzir o ambiente e executar o projeto em qualquer máquina. O passo a passo foi validado por outro integrante do grupo.

6.1. Pré-requisitos

- Python 3.10 ou superior instalado (recomendado: 3.11 ou 3.12);
- Acesso à rede da FIAP ou VPN ativa (necessário para conectar ao banco Oracle durante o treino);
- Editor de código (VS Code recomendado) ou terminal de sua preferência;
- Node.js instalado apenas se desejar executar o frontend React junto com a API.

6.2. Instalação e Execução

A Tabela 4 descreve o passo a passo completo para instalação e execução:

Tabela 4 – Passo a passo de execução local

Nº	Ação	Comando / Detalhe
1	Extrair o projeto	Extraia o arquivo IA.zip em uma pasta local. Exemplo: C:\Projetos\IA
2	Abrir terminal	Abra o VS Code e use o terminal integrado (Ctrl + `) ou o Prompt de Comando
3	Navegar até a pasta	cd C:\Projetos\IA (ou o caminho onde extraiu)
4	Criar ambiente virtual	python -m venv venv
5a	Ativar venv (Windows)	.\venv\Scripts\activate
5b	Ativar venv (Mac/Linux)	source venv/bin/activate
6	Instalar dependências	pip install -r requirements.txt
7	Verificar conexão Oracle	Confirme que está na rede FIAP ou com VPN ativa
8	Executar pipeline completa	python run.py
9	Aguardar conclusão	O terminal exibirá as 3 etapas: gerar dados, treinar falta, treinar arrecadação
10	API disponível	Acesse http://127.0.0.1:8000/health para confirmar que a API está no ar

6.3. Opções de Execução

O script `run.py` aceita os seguintes argumentos opcionais:

- **`python run.py`** — executa a pipeline completa: gera dados do Oracle, treina os dois modelos e sobe a API;
- **`python run.py --only-api`** — sobe apenas a API sem retreinar. Use quando os modelos `joblib` já existem na pasta `models/`;
- **`python run.py --only-train`** — apenas treina os modelos sem subir o servidor Flask;
- **`python app.py`** — sobe a API diretamente, equivalente ao `--only-api`.

6.4. Testando os Endpoints Manualmente

Com a API em execução, os endpoints podem ser testados via navegador (GET) ou Postman (POST). Exemplos de requisições:

Endpoint `/predict/falta` — método POST, corpo JSON:

```
{
  "distanciaKm": 15,
  "rendaFamiliar": 900,
  "faltasAnteriores": 2,
  "diasAteConsulta": 7,
  "turnoConsulta": "NOITE"
}
```

Endpoint `/predict/arrecadacao` — método POST, corpo JSON:

```
{
  "metaValor": 20000,
  "duracaoDias": 30,
  "mesInicio": 5,
  "campanhasAnteriores": 4
}
```

O endpoint GET `/health` retorna um JSON com o status da API e o estado de carregamento dos dois modelos, sendo útil para verificar rapidamente se o serviço está operacional antes de enviar predições.

7. CONSIDERAÇÕES FINAIS

A Sprint 4 conclui a camada de Inteligência Artificial do projeto De Novo Não! integrando efetivamente o modelo preditivo ao ecossistema completo da ONG Turma do Bem: banco Oracle como fonte de dados reais, API Python como camada de inferência, e ERP React como interface de consumo.

O sistema preditivo implementado atende a todos os critérios do enunciado: pipeline de pré-processamento com encoding de categorias, normalização e engenharia de features; treino e comparação de pelo menos três modelos distintos com validação cruzada; seleção automática do melhor modelo por métrica objetiva (F1 para classificação, MAE para regressão); serialização com joblib em objeto único e portátil; e deploy como API Flask com os endpoints /predict e /health, além de endpoints complementares.

O modelo de previsão de falta apresenta bom desempenho ($F1 = 0,8571$) com os dados disponíveis, sendo capaz de identificar pacientes de alto risco antes da consulta. O modelo de arrecadação tem margem de melhora à medida que o banco acumule mais campanhas encerradas, fato esperado e documentado neste relatório.

Ambos os modelos são retreináveis automaticamente via endpoint /retreinar, sem necessidade de reinicialização da API. O projeto está preparado para crescer: a pipeline é reproduzível seguindo o passo a passo da Seção 6, os arquivos. joblib são portáteis entre máquinas, e a API é independente de ambiente de execução.